

Scalable Technologies for Data Analysis

Incremental Processing and Streaming

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026

Incremental Processing and Streaming

When batch processing is not enough

- By the end of this class you should be able to
 - Distinguish between **batch**, **incremental**, and **streaming** processing
 - Explain the main patterns of incremental pipelines: append, deduplication, upserts, and window repair
 - Describe CDC (change data capture) and its role in analytical data pipelines
 - Explain event time, processing time, and the challenge of late data
 - Understand basic delivery semantics in streaming systems
 - Describe how Kafka supports event-driven data architectures
 - Choose the right processing paradigm based on freshness, cost, and complexity

The plan

1. Intro to modern Data Platforms and scalability
2. Contract-Driven Design and Data Flow Modeling
3. Storage + table formats
4. Processing patterns + optimization
5. Orchestration + production reliability
6. Serving: SQL/APIs/semantic layer
7. Quality + observability + cost
8. Incremental + streaming 
9. ML integration
10. Governance + evolution

Previously...

- In the previous classes, we...
 - Designed data platforms around data products, contracts, and Bronze / Silver / Gold layers
 - Learned how to store and organize analytical data efficiently using object storage, Parquet, partitioning, and Iceberg
 - Built scalable transformations and reliable workflows with Trino and Flyte
 - Discussed serving, quality, observability, idempotency, retries, and backfills as foundations for production-grade pipelines
- Today we...
 - Move from periodic recomputation to data that changes continuously
 - Introduce incremental processing as a way to update only what changed
 - Explore CDC and streaming fundamentals, including event time, processing time, and late data
 - Understand how Kafka fits into event-driven data architectures
 - Learn how to choose between batch, incremental, and streaming based on freshness, cost, and complexity

Relevant concepts recap

- Medallion layers
- Data contracts
- Grain and keys
- Determinism and reproducibility
- Logical time
- Partitioning and Iceberg snapshots
- Idempotency, retries, and backfills
- Freshness and validation gates



Relevant concepts recap

- Medallion layers: Bronze preserves raw data, Silver cleans and conforms it, Gold serves stable business outputs
- Data contracts: Define schema, keys, grain, semantics, and freshness expectations
- Grain and keys: Clarify what one row means and how it is uniquely identified
- Determinism and reproducibility: Same inputs and logic should produce the same result, and we should be able to re-create it
- Logical time: Pipelines should process explicit data intervals, not depend on “now()”
- Partitioning and Iceberg snapshots: Enable partial recomputation, repair windows, and controlled publishing
- Idempotency, retries, and backfills: Make reruns, replays, and historical repairs safe
- Freshness and validation gates: Define when data is acceptable and whether it is safe to publish



Introduction

From reliable batch pipelines to continuously changing data



From reliable batch pipelines to continuously changing data

- In the previous classes, we designed reliable, reproducible, and scalable data pipelines
- We covered
 - Data contracts, Bronze / Silver / Gold
 - Storage and table formats in the lakehouse
 - Scalable processing and optimization
 - Orchestration, idempotency, retries, and backfills
 - Serving, quality, and freshness
- So far, the dominant mental model has been
 - Bounded data
 - Scheduled runs
 - Recompute a defined data interval
- Today we extend that model to data that changes continuously and **may arrive late, out of order, or as change events rather than static snapshots**
- All the principles we discussed previously continue to hold, but now we must apply them to more dynamic data

Why this matters?

- When we receive data that is out of order or late, we might have already recomputed some partitions
- The easy solution is to do a full recomputation, which works well when
 - Data volumes are moderate
 - Updates are infrequent
 - Latency expectations are relaxed
- But it becomes problematic when
 - Source data changes frequently
 - Historical volume keeps growing
 - Consumers expect fresher outputs
 - Recomputing everything becomes too slow or too expensive
- This creates a new architectural question: should we **recompute everything**, only **what changed**, or **react continuously to events**?
- The challenge is not only speed, it's correctness under continuous change
- Also keep in mind previous aspects we have already covered
 - Cost and scalability
 - Logical time and backfills
 - Quality and freshness expectations
 - ...

Batch

Incremental

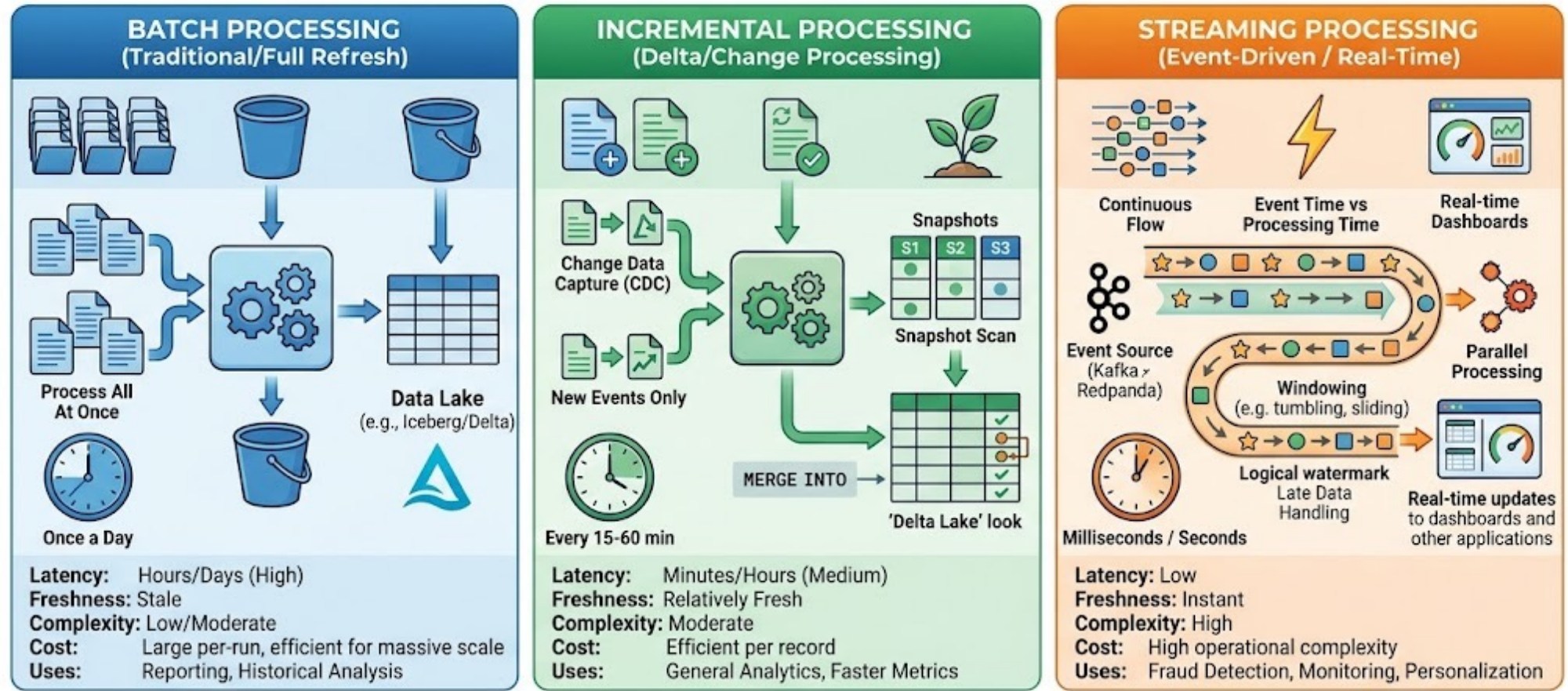
Streaming

On today's menu

- Today we will focus on
 - Incremental processing in analytical pipelines
 - CDC (change data capture) as a source of change
 - Streaming fundamentals
 - Event-driven architectures with Kafka
 - How to choose between batch, incremental, and streaming
- In our stack, this means
 - Kafka for event streams
 - MinIO + Iceberg for durable analytical storage
 - Trino for analytical processing and serving
 - Flyte for scheduled workflows, repairs, and backfills
- This will not be a full streaming-engine implementation class
- The goal is to build
 - The right conceptual vocabulary
 - The right architectural decision framework
 - A clear understanding of how these ideas fit into our existing data platform (or into other similar ones)

A visual introduction (by Nano Banana 2)

COMPARISON OF DATA PROCESSING METHODS: BATCH, INCREMENTAL, AND STREAMING



CDC – Change Data Capture (quick intro)

- Change Data Capture is a way of capturing changes in a source system as they happen
- Instead of repeatedly copying the full table, CDC captures only the relevant changes, such as inserts, updates or deletes
- In practice, it is often implemented by reading the database transaction log and publishing change events
- Example flow
 - Source DB: orders
 - CDC connector reads the transaction log
 - Changes are published to a Kafka topic such as orders_cdc
 - Example events:
 - new row inserted
 - order status updated
 - row deleted or cancelled
 - Downstream pipelines consume these events incrementally instead of re-reading the full table
- This makes it possible to build more efficient incremental pipelines
- What we had until now were Snapshots, which contain the **current state** of the database. CDC captures **the changes** that produced that state

Incremental processing

Batch vs. incremental

- When data changes, the simplest pipeline strategy is often
 - Read the required input data
 - Recompute the target dataset from scratch
 - Publish the new result
- This works well when
 - Datasets are still manageable
 - Refreshes are infrequent
 - Latency requirements are relaxed
 - The cost of recomputation is acceptable
- This has been the dominant mental model in many of our examples and tutorials so far
 - Bounded input
 - Explicit data interval
 - Deterministic rebuild of a partition or table
- If these principles hold, we don't need to do incremental processing just because it is cool. If it works well in batch, we leave it in batch
 - It is simpler to reason about
 - It is simpler to implement
 - It is easier to validate

Batch vs. incremental

- Full recomputation becomes problematic when
 - Data volume keeps growing
 - Updates become more frequent
 - Consumers expect fresher outputs
 - Historical rebuilds become expensive
- Typical symptoms
 - Daily pipelines start taking too long
 - Recomputing the full table scans far more data than necessary
 - Late data forces repeated reprocessing of large windows
 - Backfills collide with normal production workloads
 - Costs increase even when only a small fraction of data actually changed
- In other words
 - The system keeps paying for what stayed the same, not only for what changed
- At scale, full refresh often becomes correct but wasteful

Incremental Processing

- Incremental processing means **processing only new, changed, or affected data** instead of rebuilding everything every time
- The core idea is “simple”: if only part of the source changed, only part of the target may need to change
- Typical strategies include:
 - Append only the new records
 - Update or merge changed rows
 - Recompute only the affected partitions or time windows
 - Repair recent windows to incorporate late arrivals
- Main expected benefits
 - Lower compute cost
 - Faster refreshes
 - Less unnecessary data movement
 - Better alignment with freshness requirements
- Incremental is not just an optimization trick as it introduces **new correctness questions about identity, ordering, late data, and safe publication**
- In a few words, incremental processing is about **selective recomputation**
- The benefit is efficiency, but the price to pay is more state and more rules

Why it is not so “*simple*”

- If we do not recompute everything, we must know
 - What changed
 - What output is affected
 - How to update it safely
- This immediately raises hard questions:
 - How do we identify the same row or event?
 - What do we do with duplicates?
 - What happens if data arrives late?
 - Can the same run be retried safely?
 - How do we repair historical outputs without corrupting them?
- So, incremental processing depends on concepts we already know
 - Keys and grain
 - Logical time
 - Idempotency
 - Partition-aware recomputation
 - Atomic publish
- This is why incremental processing is not just “smaller batch”: it is a correctness and state-management problem

Incremental processing in the lakehouse

What does incremental processing actually look like in the lakehouse?

Incremental processing in the lakehouse

- Incremental processing means updating only the data that is new, changed, or affected, instead of rebuilding the entire dataset every time
- In a lakehouse, this usually means
 - Appending new records
 - Replacing affected partitions or windows
 - Merging changed rows into an existing table
 - Repairing recent windows to incorporate late data
- The main goal is to reduce
 - Unnecessary scans
 - Unnecessary recomputation
 - Latency to publish updated outputs
- But incrementality only works safely if we can define
 - What changed
 - What output is affected
 - How to publish the update deterministically

Pattern 1: append-only ingestion

- The simplest incremental pattern is **append-only**
- New data is added without modifying previously written records
- This is especially natural in Bronze, where the goal is to preserve source fidelity and provenance
- Typical examples
 - Application events
 - Clickstream logs
 - Raw CDC events
 - Daily ingestion batches
- Main advantages
 - Simple to implement
 - Easy to replay
 - Good auditability
- Main limitations
 - Duplicates may accumulate
 - Corrections are not resolved automatically
 - Consumers usually should not query raw append-only data directly

Pattern 2: replace affected partitions or windows

- A common incremental strategy is to recompute only the affected partition or time window
- Instead of rebuilding the whole table, we rebuild only one day, one hour, one partition, one bounded historical window...
- This works well when
 - Data is naturally partitioned by time
 - Corrections are localized
 - We want deterministic reruns
- Typical uses
 - Rebuild date = 2026-03-10
 - Recompute the last 3 days to capture late arrivals
 - Repair a historical window after a bug fix
- Main advantages
 - Simpler than row-level updates
 - Aligns well with backfills and logical time
 - Safer for idempotent publishing
- Main limitation: even a small correction may require rebuilding the full partition, so efficiency gains are reduced...

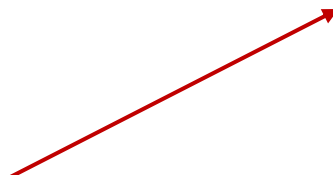
Pattern 3: upserts and MERGE

- Sometimes we need to update the current state of a table, not just append or rebuild a whole partition
- In those cases, we use **upserts**. This may mean
 - Insert new rows
 - Update existing rows
 - Sometimes delete rows that are no longer valid
- This is often expressed through MERGE semantics
- Upserts require
 - A stable key
 - Clear row identity
 - Explicit rules for conflict resolution
- Typical examples
 - Customer profile tables
 - Latest order status
 - Current inventory state
- Main risk: if keys or semantics are unclear, incremental merges can silently produce duplicates or wrong state

Pattern 3: upserts and MERGE

- Sometimes we need to update the current state of a table, not just append or rebuild a whole partition
- In those cases, we use **upserts**. This may mean
 - Insert new rows
 - Update existing rows
 - Sometimes delete rows that are no longer valid
- This is often expressed through MERGE semantics
- Upserts require
 - A stable key
 - Clear row identity
 - Explicit **rules for conflict resolution**
- Typical examples
 - Customer profile tables
 - Latest order status
 - Current inventory state
- Main risk: if keys or semantics are unclear, incremental merges can silently produce duplicates or wrong state

Which rules are
encoded here?



```
3 MERGE INTO silver.customers_current AS tgt
4 USING silver.customers_changes AS src
5 ON tgt.customer_id = src.customer_id
6
7 WHEN MATCHED
8     AND src.is_deleted = true THEN
9     UPDATE SET
10         is_active = false,
11         updated_at = src.updated_at
12
13 WHEN MATCHED
14     AND src.updated_at > tgt.updated_at THEN
15     UPDATE SET
16         customer_name = src.customer_name,
17         email = src.email,
18         country = src.country,
19         is_active = true,
20         updated_at = src.updated_at
21
22 WHEN NOT MATCHED
23     AND src.is_deleted = false THEN
24     INSERT (
25         customer_id,
26         customer_name,
27         email,
28         country,
29         is_active,
30         updated_at
31     )
32     VALUES (
33         src.customer_id,
34         src.customer_name,
35         src.email,
36         src.country,
37         true,
38         src.updated_at
39     );
```

Deduplication and late data

- Incremental pipelines must deal with the fact that data may be duplicated, retried, delayed or out of order
- This makes deduplication a central concern, especially in Silver
- Deduplication depends on
 - A stable event or business key
 - Rules for deciding which record wins
 - Deterministic handling of retries and replays
- Another challenge is late data
 - Yesterday's partition may still change today
 - A metric published this morning may need repair tomorrow
- A common solution is a sliding repair window
 - Each run recomputes not only the current partition but also the most recent N partitions or days
- This improves correctness without forcing full historical recomputation

How this maps to our stack

- In our stack
 - MinIO stores the data files
 - Iceberg manages table metadata, snapshots, and atomic commits
 - Trino reads and writes analytical tables
 - Flyte orchestrates scheduled runs, repairs, and backfills
- A typical incremental flow is
 - Ingest new raw data into Bronze
 - Clean, deduplicate, and normalize into Silver
 - Publish stable business-facing outputs into Gold
- Operationally, this means
 - Rebuild only the affected windows when possible
 - Use stable keys for merges and deduplication
 - Publish outputs only after validation
 - Keep repairs and backfills parameterized by logical time
- So, incremental processing in the lakehouse is not one single technique
 - It is a combination of append, replace, merge, and repair patterns applied safely

CDC

Change Data Capture

Change Data Capture

- CDC captures changes in a source system instead of repeatedly copying its full current state
- Typical change events include
 - INSERT
 - UPDATE
 - DELETE
- This makes CDC a natural fit for incremental pipelines
- Instead of asking: “What does the table look like now?”
- CDC asks: “What changed since the last point in time?”
- In practice, CDC is often used to move changes from operational databases into the data platform as a stream of events

Snapshot extraction vs. CDC

- There are two common ways to extract data from a source table
- Snapshot extraction
 - Reads the table as it exists at a certain moment
 - Gives the current state
 - Is simpler to reason about
 - May require repeatedly re-reading large amounts of unchanged data
- CDC
 - Captures the sequence of changes that produced the current state
 - Transfers only what changed
 - Supports fresher and more incremental updates
 - Introduces more complexity around ordering, duplicates, and deletes
- In other words
 - Snapshot extraction is **state-oriented**
 - CDC is **change-oriented**

Example: CDC from an orders table

- Suppose an operational database has a table called orders
- During the day
 - A new order is created
 - An existing order changes status from pending to paid
 - A cancelled order is deleted or marked as cancelled
- With snapshot extraction, we would re-read the table periodically
- With CDC, we capture only the changes, for example
 - `INSERT order_id=101, customer_id=77, status='pending', amount=45.00`
 - `UPDATE order_id=101, status='paid'`
 - `DELETE order_id=55`
- These change events can then be published to a Kafka topic
- Downstream pipelines consume them incrementally instead of re-reading the full table

How CDC fits into our stack

- In our context, a typical CDC flow is
 - The source database generates row-level changes
 - A CDC connector captures those changes
 - Changes are published to Kafka
 - Our platform consumes those events into Bronze
 - Silver deduplicates, normalizes, and reconstructs the trusted state
 - Gold serves business-ready tables and metrics
- This means CDC does not replace the lakehouse: it provides a more incremental and event-driven input into the lakehouse
- CDC is especially useful when
 - Source tables change frequently
 - Full refresh is too expensive
 - Freshness expectations are higher
- But CDC also requires clear rules for
 - Identity
 - Ordering
 - Deduplication
 - Delete handling

Streaming fundamentals

Streaming

- In this context of this class, streaming is a processing model in which data is handled as a continuous flow of events, rather than as a fixed, bounded dataset
- In batch processing
 - The input is finite
 - The job starts and ends
 - The full result is produced after the run
- In streaming
 - Events keep arriving over time
 - The input is potentially unbounded
 - Results may be updated continuously or in near real time
- Typical examples
 - Clicks from a website
 - Application logs
 - IoT sensor readings
 - Payment or order events
- So, streaming is not only “faster batch”: it is a different way of reasoning about time, state, and continuous updates

Bounded vs. unbounded data

- A useful distinction is between bounded and unbounded data
- Bounded data
 - Has a clear beginning and end
 - Can be processed as a finite dataset
 - Fits naturally into batch workflows
- Unbounded data
 - Keeps growing over time
 - Has no natural “end”
 - Fits naturally into streaming workflows
- Examples
 - A CSV file exported yesterday -> bounded
 - A Kafka topic with user activity events -> unbounded
- This matters because many analytical questions still require us to produce finite outputs from unbounded input
- To do that, streaming systems use concepts such as
 - Time
 - Windows
 - Watermarks
 - Stateful processing

Event time vs. processing time

- In streaming systems, there are at least two important notions of time
 - Event time: when the event actually happened in the real world. Example: when the order was placed or when the click occurred
 - Processing time: when the system observes and processes the event. Example: when the message reaches Kafka or is consumed by a job
- These two times are not always the same
- An event may arrive late, out of order or after a temporary network or system delay
- This matters because analytical correctness usually depends more on event time than on processing time
- Example
 - A purchase made at 08:59 may only be processed at 09:07
 - If we group by processing time, we may assign it to the wrong window

Windows, watermarks, and late data

- Because streaming input is unbounded, we often compute results over windows
- A window is a finite slice of the event stream, such as
 - One minute
 - One hour
 - One day
 - A sliding interval over the last N minutes
- But events may arrive late, so the system must decide
 - When a window is ready to be emitted
 - How long to wait for delayed events
 - Whether late events should update previous results
- A watermark is a signal used to estimate how far event time has progressed
- Informally, it says “we believe most events up to this point have already arrived”
- This helps the system decide when to
 - Emit windowed results
 - Close windows
 - Handle late data differently

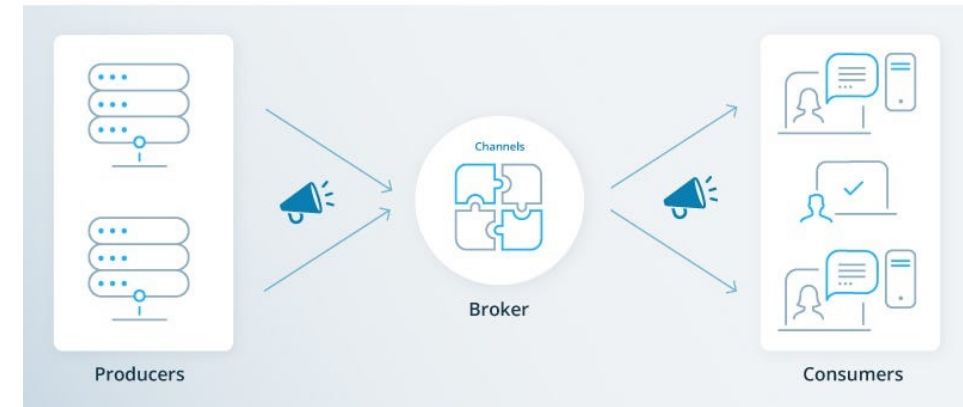
Event-driven architecture overview

(in our stack)

What is an event-driven architecture?

- In an event-driven architecture, systems communicate by producing and consuming events
- An event is a record that something happened, for example
 - An order was created
 - A payment was confirmed
 - A shipment was delivered
 - A user clicked a button
- Instead of periodically asking a source system for its current state, downstream systems react to the events it emits
- This makes event-driven architectures a natural fit for
 - Incremental updates
 - Streaming pipelines
 - Loosely coupled systems
- The key idea is
 - Producers emit events
 - Consumers react to them asynchronously

Event-Driven Architecture



Where Kafka fits

- Kafka is a distributed event streaming platform
- In practice, it is often used as the central layer that transports and stores streams of events between producers and consumers
- A typical flow is
 - A source system produces events
 - Events are written to a Kafka topic
 - One or more consumers read those events independently
- This is useful because Kafka
 - Decouples producers from consumers
 - Supports high-throughput event ingestion
 - Allows multiple consumers to reuse the same event stream
 - Retains events for some period, enabling replay
- In this sense, Kafka is not the analytical platform itself: it is the event transport and buffering layer between operational systems and downstream consumers

Where Kafka fits

- In our stack, Kafka fits before the lakehouse layers
- A simplified flow is
 - Source systems produce events or CDC changes
 - Those events are published to Kafka topics
 - Ingestion consumers write raw events into Bronze on MinIO / Iceberg
 - Silver cleans, deduplicates, and normalizes the data
 - Gold publishes stable business-ready outputs
 - Trino serves analytical access to the lakehouse
 - Flyte orchestrates scheduled repairs, rebuilds, and downstream workflows
- This means
 - Kafka handles event transport
 - The lakehouse remains the durable analytical system of record
 - Flyte still manages batch and incremental operational workflows around the lakehouse

Where Kafka fits

- Introducing Kafka does not mean the whole platform must become fully streaming
- A common pattern is
 - Streaming ingestion into Bronze
 - Incremental or micro-batch processing into Silver
 - Batch or incremental publication into Gold
- This hybrid approach is often practical because it preserves
 - Clear contracts
 - Repairability
 - Backfills
 - Deterministic publication
- In other words
 - Event-driven ingestion increases freshness
 - But downstream layers may still use bounded, controlled processing models
- So, the real architectural choice is not streaming everywhere
- It is **where is streaming justified, and where is batch or incremental still the better choice?**

Batch vs incremental vs streaming



When is batch enough?
When is incremental the better compromise?
When is streaming justified

Batch vs incremental vs streaming

- There is no universally best processing model
- Batch, incremental, and streaming are all valid processing models
- The right choice depends on the use case, not on which approach seems more modern
- In practice, the goal is to choose the model that provides
 - Acceptable freshness
 - Acceptable cost
 - Acceptable operational complexity
 - Acceptable correctness and reliability
- A more advanced architecture is not automatically a better architecture
- The key question is: What is the simplest processing model that still satisfies the requirements?

When batch is enough

- Batch is often the best choice when
 - Data can be processed in clearly bounded intervals
 - Freshness requirements are relaxed
 - Full or windowed recomputation is still affordable
 - Simpler operation and easier debugging are more important than very low latency
- Typical examples
 - Daily KPI tables
 - Weekly reports
 - Historical feature generation for ML training
 - Overnight reconciliations
- Main strengths
 - Simpler logic
 - Easier validation and reproducibility
 - Easier backfills and historical repairs
- Main limitation: data is only as fresh as the next scheduled run

When incremental is the better compromise

- Incremental processing is useful when
 - Full recomputation is becoming too expensive
 - Only a small fraction of data changes between runs
 - Consumers need fresher outputs than classic batch can provide
 - We can identify affected rows, partitions, or windows reliably
- Typical examples
 - Daily tables with late-arriving events
 - Current-state tables built from frequent source updates
 - Repair of the last N days instead of full historical rebuild
 - CDC-driven updates into analytical tables
- Main strengths
 - Lower cost than full recomputation
 - Better freshness
 - Still compatible with controlled, scheduled execution
- Main limitation: more complex correctness rules around keys, deduplication, and late data

When streaming is justified

- Streaming becomes justified when
 - Events arrive continuously at significant velocity
 - Consumers need near-real-time updates
 - Waiting for the next scheduled run is not acceptable
 - The business value of freshness justifies the extra complexity
- Typical examples
 - Operational monitoring dashboards
 - Fraud or anomaly detection
 - Real-time user activity tracking
 - Alerting systems
- Main strengths
 - Lowest latency
 - Natural fit for continuous event flows
 - Enables fast reaction to new events
- Main limitations
 - More difficult reasoning about time and state
 - More complex recovery and correctness guarantees
 - Higher operational complexity

A practical decision framework

- When choosing between batch, incremental, and streaming, evaluate
 - Freshness: how quickly must the output be updated?
 - Volume and change rate: how much data changes between runs?
 - Cost: is full recomputation wasteful?
 - Operational complexity: can the team operate a more complex solution reliably?
 - Correctness requirements: how sensitive is the use case to duplicates, late data, or partial failure?
 - Repairability: how easily can we replay, backfill, or correct historical outputs?
- A practical rule of thumb
 - Batch when latency tolerance is high and simplicity matters most
 - Incremental when recomputing everything is wasteful but bounded execution is still desirable
 - Streaming when freshness requirements justify continuous processing
- In practice, many real systems are hybrid
 - Streaming ingestion
 - Incremental repair in Silver
 - Batch or incremental publication in Gold

Conclusion

- Incremental processing exists because full recomputation becomes too expensive, too slow, or too wasteful as data volume and update frequency grow
- In practice, incrementality in the lakehouse is implemented through patterns such as append-only ingestion, partition or window repair, upserts / MERGE or deduplication
- CDC is an important source of incremental change, because it captures inserts, updates, and deletes instead of repeatedly copying full table state
- Streaming is a different processing model for unbounded event flows, where reasoning about event time, processing time, windows, watermarks, and late data becomes essential
- Kafka fits as the event transport layer in an event-driven architecture, while the lakehouse remains the durable analytical system of record
- In practice, many platforms are hybrid
 - Streaming or CDC-based ingestion
 - Incremental processing in intermediate layers
 - Batch or incremental publication for stable analytical outputs
- There is no universally best model
 - Batch favors simplicity and reproducibility
 - Incremental reduces unnecessary recomputation
 - Streaming improves freshness, but adds complexity
- The right choice depends on the required balance between
 - Freshness
 - Cost
 - Correctness
 - Operational complexity
 - Repairability
- Main takeaway: The goal is not to make everything real-time, but to choose the simplest processing model that satisfies the required freshness and correctness

Scalable Technologies for Data Analysis

Incremental Processing and Streaming

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026